

Angewandte KI & KI-Engineering (AKE)

Block A – Vertiefung Deep Learning

Lab 1: MLP für MNIST

Einführung in Deep-Learning

Wintersemester 2026/27

Organisatorisches – Lab Woche 1

Termin	KW 41 (08.10), 2-SWS-Lab-Block im Anschluss an den Theorieblock
Skript	<i>Vom Perzeptron zum Deep Network</i> , insbesondere Abschn. 3–8 (Theorie) und 9 (Lab)
Material	Kursrepository, Ordner <code>labs/woche01/</code> : <code>setup_check.ipynb</code> , <code>lab01_starter.ipynb</code> , <code>results_referenz/</code> (Referenz-Trainingslogs, falls eigene Läufe abbrechen)
Umgebung	Python 3.11+, PyTorch (CPU-Build), torchvision, Jupyter, matplotlib – aus der versionierten Environment-Datei (<code>uv sync</code> bzw. <code>conda env create -f environment.yml</code>)
Zeitplan	45 Min. Aufgabe 0 (Setup ist der kritische Pfad, nicht die Rechenzeit) · 45 Min. Aufgaben 1–2 · Rest und Aufgaben 3–4 als Selbststudium

Aufgaben

Aufgabe 0: Umgebung einrichten und prüfen (Pflicht, ohne Abgabe)

1. Setup-Anleitung im Anhang durcharbeiten
2. `setup_check.ipynb` ausführen. Das Notebook prüft `torch.__version__`, Thread-Zahl, verfügbaren RAM, den MNIST-Bezug und misst die Laufzeit von 100 Trainingsschritten des Basis-MLP (Batch 64). Erwartung: hochgerechnet ≤ 45 s pro Epoche auf 4 Kernen.
3. Alle Prüfungen grün: weiter mit Aufgabe 1. Sonst: Notebook-Protokoll vorzeigen. Typische Stolpersteine: Proxy (`pip config set global.proxy`), Python-Version < 3.11 , fehlende Adminrechte (dann `uv` statt `conda`).

Aufgabe 1: Baseline – MLP 784–128–128–10

Trainieren Sie das MLP aus Listing 1 des Skripts (zwei verdeckte Schichten à 128 Einheiten, ReLU, Logit-Ausgabe) mit Adam ($\alpha = 10^{-3}$), Batch 64, 5 Epochen auf den 54 000 Trainingsbildern. Validierung auf 6 000 zurückgehaltenen Bildern, Testgenauigkeit *einmalig* am Ende.

1. Vervollständigen Sie die Klasse MLP im Starter-Notebook (variable Tiefe, Breite, Aktivierung, optional Dropout). Prüfen Sie die Parameterzahl: 118 282 (Bsp. 3.2).
2. Debug-Protokoll, Schritt 1 (Abschn. 8): Verlust beim ersten Batch vor dem Training ausgeben. Erwartung $\approx \ln 10 = 2,30$.
3. Trainieren Sie 5 Epochen und plotten Sie Trainings- und Validierungsverlust je Epoche. Erwartung: 97–98 % Testgenauigkeit.

Welchem der vier Grundmuster der Vorlesung entspricht Ihr Verlauf? Woran erkennen Sie, ob längeres Training noch etwas bringen würde? Warum darf die Ausgangsschicht kein Softmax enthalten (Abschn. 3.4)?

Aufgabe 2: Architektur – Vanishing Gradients sichtbar machen

Bauen Sie ein 10-schichtiges Netz (10 verdeckte Schichten à 100 Einheiten), einmal mit Sigmoid und einmal mit ReLU, jeweils SGD mit $\epsilon = 0,1$.

1. Führen Sie *einen* Vorwärts- und Rückwärtsdurchlauf auf dem ersten Batch aus und messen Sie

die Gradientennorm der ersten Schicht: `model.net[1].weight.grad.norm()`. Erweitern Sie die Messung auf alle Linear-Schichten und plotten Sie die Norm über der Schichtnummer (log-Skala).

2. Trainieren Sie beide Netze 3 Epochen auf dem 10 000er-Subset `dl10k`. Zeichnen Sie $\ln 10$ als Referenzlinie ein.

Erklären Sie das Verhältnis der Gradientennormen zwischen erster und letzter Schicht mit Gl. (29): Welcher Faktor in $J_l = \text{diag}(\varphi'(\mathbf{a}_{l+1})) \mathbf{W}_{l+1}$ dämpft bei Sigmoid, welcher bei ReLU? Welche Zeile von Tab. 5 beschreibt den Sigmoid-Verlauf? Nennen Sie zwei Gegenmaßnahmen aus Abschn. 7.2 und ordnen Sie sie dem jeweiligen Faktor zu.

Aufgabe 3: Optimierer $\times$ Lernrate

Für SGD, SGD+Momentum ($\alpha = 0,9$) und Adam: Lernraten $\{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$, je 2 Epochen, Baseline-Architektur, gleicher Seed für alle 12 Läufe. **Rechenbudget:** Der Sweep läuft auf dem 10 000er-Subset (`dl10k`), validiert wird auf dem vollen Validierungssplit ($\approx 2\text{--}3$ Min. auf 4 Kernen).

1. Protokollieren Sie den Validierungsverlust nach 2 Epochen als 3×4 -Tabelle; markieren Sie Läufe mit NaN/inf oder Verlust $\geq \ln 10$ als *divergiert*.
2. Plotten Sie die Validierungskurven (ein Diagramm je Optimierer, eine Linie je Lernrate, log-Skala).

Welche Kombination erreicht den geringsten Validierungsverlust, welche divergiert – und warum (Abb. 11, Gl. 18, Abb. 13)? Über wie viele Zehnerpotenzen der Lernrate ist jeder Optimierer „brauchbar“? Was folgt daraus für Schritt 3 des Debug-Protokolls?

Aufgabe 4: Regularisierungs-Ablation bei Datenknappheit

Reduzieren Sie das Training auf 1 000 Beispiele (`dl1k`) und trainieren Sie die Baseline-Architektur 50 Epochen mit Adam (10^{-3}) in vier Varianten: *keine Regularisierung* / *Weight Decay* 10^{-3} / *Dropout* 0,5 / *Early Stopping* (patience 5, Parameterstand am Validierungsminimum behalten).

1. Plotten Sie Trainings- und Validierungsverlust aller Varianten in einem Diagramm, die Validierungsgenauigkeit in einem zweiten.
2. Tabelle: trainierte Epochen, bester Validierungsverlust, Testgenauigkeit (je Variante genau eine Testauswertung).
3. Optional: eine fünfte Variante Ihrer Wahl (z. B. Dropout + Early Stopping oder AdamW mit größerem λ).

Beschreiben Sie die Schere der unregulisierten Variante (Abb. 14) – warum steigt der Validierungsverlust, während die Validierungsgenauigkeit kaum fällt (Tab. 5, letzte Zeile)? Warum wirkt Weight Decay 10^{-3} bei Adam kaum (Abschn. 6.2)? Warum liegt bei Dropout der Validierungsverlust zeitweise unter dem Trainingsverlust? Welche Kombination empfehlen Sie – begründet mit der Reihenfolge aus Tab. 4?

Aufgabe 5: Optional – Backpropagation in NumPy

Implementieren Sie `backward_np` für das einschichtige MLP aus Bsp. 4.1 (ReLU, Softmax + Cross-Entropy, Mittelung über den Batch) und verifizieren Sie alle vier Gradienten per zentraler finiter Differenzen ($\varepsilon = 10^{-5}$) auf einem winzigen Netz in `float64`. Ziel: maximale Abweichung $< 10^{-7}$. Zählen Sie, wie viele Vorwärtsdurchläufe die numerische Prüfung benötigt, und vergleichen Sie mit Abschn. 4.3.

Hinweise

In der Praxis

Debug-Protokoll vor jedem Experiment (Skript Abschn. 8): (1) Verlust beim ersten Batch $\approx \ln C$; (2) einen Minibatch überfitten – Verlust muss gegen 0 gehen, sonst ist es ein Codeproblem; (3) Lernrate über Zehnerpotenzen rastern, alles andere fest; (4) erst dann Architektur und Regularisierung – eine Variable zugleich. Loggen Sie jeden Lauf (Seed, Konfiguration, Kurven) – das Starter-Notebook schreibt `results/*.json`.

- **Testsplit-Regel:** Der Testsplit dient je Aufgabe genau einer finalen Bewertung. Wer ihn zur Auswahl von Hyperparametern nutzt, misst eine zu optimistische Generalisierung (Abschn. 6.1).
- **Reproduzierbarkeit:** `torch.manual_seed(0)` vor *jedem* Modell – sonst vergleichen Sie in Aufgabe 3 und 4 verschiedene Startpunkte statt verschiedener Hyperparameter.
- **`model.eval()` / `torch.no_grad()`:** in der Evaluationsschleife Pflicht; sonst bleibt Dropout aktiv und die Aufzeichnung des Graphen kostet Speicher (Abschn. 4.6, 6.4).
- **Fallback:** Bricht ein Lauf ab oder dauert er auf Ihrer Hardware zu lange, laden Sie das passende JSON aus `results_referenz/` und bearbeiten Sie die Interpretation damit. Kennzeichnen Sie das im Notebook.
- **Repository-Struktur:** Legen Sie jetzt `data/`, `src/`, `notebooks/`, `configs/`, `results/` an (Abschn. 9.3) – das Trägermodell aus dem nächsten Lab und alle MLOps-Labs bauen darauf auf.

Anhang: Setup-Anleitung für den eigenen Rechner

Alle Basis-Labs laufen vollständig auf Ihrem eigenen Laptop – ohne GPU und ohne Adminrechte. Sie brauchen Python (ab 3.11), die Paketverwaltung `uv`, Git und das Kursrepository. Ablauf: `uv` installieren → Repository klonen → `uv sync` → `jupyter notebook` starten → `setup_check.ipynb`.

Komponente	Mindestens	Hinweis
Prozessor	4 CPU-Kerne	Mehr Kerne verkürzen die Trainingszeiten.
Arbeitsspeicher	8 GB RAM	Mit ≥ 16 GB ist im Lab <i>LLM-Anpassung</i> optional ein größeres Modell möglich.
Freier Speicher	15 GB	Umgebung, Datensätze, Modellgewichte, später ein Docker-Image (≈ 1 GB).
Betriebssystem	Windows 10/11, macOS, Linux	
GPU	nicht erforderlich	Apple Silicon (MPS) oder NVIDIA (CUDA) beschleunigen, sind aber nie Voraussetzung.
Adminrechte	nicht erforderlich	Ausnahme: Docker unter Windows (Hausaufgabe, siehe Aufgabe 0).

A.1 Python-Umgebung einrichten

Die Umgebung wird mit `uv` aus der Lock-Datei des Kursrepositorys erzeugt, damit alle mit exakt denselben Paketversionen arbeiten. `uv` lädt die passende Python-Version bei Bedarf selbst herunter; eine vorhandene Python-Installation stört nicht.

1. **uv installieren.** Terminal öffnen (Windows: PowerShell) und ausführen:

```
# Windows (PowerShell)
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"

# macOS / Linux
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Danach ein *neues* Terminal öffnen und mit `uv -version` prüfen.

2. **Git installieren.** Windows: *Git for Windows* (git-scm.com); macOS: `xcode-select -install`; Linux: Paketverwaltung, z. B. `sudo apt install git`.
3. **Kursrepository klonen.** Kurzer Pfad ohne Leerzeichen und Umlaute, *nicht* in OneDrive oder iCloud (die Synchronisation sperrt Dateien während des Trainings):

```
cd ~
git clone <KURSREPO-URL> ake
cd ake
```

4. **Umgebung erzeugen.** Im Ordner `ake`:

```
uv sync
```

Legt den Ordner `.venv` an und installiert alle Pakete inklusive des CPU-Builds von PyTorch (≈ 200 MB). Installieren Sie Pakete nie zusätzlich mit `pip install` – sonst landet u. U. der CUDA-Build ($\approx 2,5$ GB) in der Umgebung.

Alternative conda: Wer bereits mit `conda` arbeitet, nutzt `conda env create -f environment.yml` und `conda activate ake`. Verwenden Sie dann Miniforge (`conda-forge`) statt Anaconda – Anacondas Standardkanäle sind für größere Organisationen lizenzpflichtig. `conda` und `uv` nicht in einer Umgebung mischen.

A.2 Daten einbinden und Notebook öffnen

Daten. Alle Datensätze liegen im Kursrepository direkt im Wurzelordner unter `data/` (z. B. `ake/data/MNIST`) und kommen mit `git clone` bzw. `git pull` automatisch auf Ihren Rechner. Die Notebooks finden `data/`

selbst, egal in welchem Unterordner von labs/ sie liegen. Den Ordner nicht umbenennen, verschieben oder darin Dateien ändern. **Notebook öffnen.** Die Labs werden ausschließlich in *Jupyter Notebook* bearbeitet, nicht in VS Code, PyCharm o. ä. – Editoren mit KI-Codevervollständigung nehmen Ihnen genau die Denkarbeit ab, um die es in den Labs geht. Im Ordner ake starten:

```
uv run jupyter notebook
```

Der Browser öffnet die Dateiübersicht des Repositorys; dort labs/woche01/setup_check.ipynb anklicken. Das Notebook nutzt automatisch die Umgebung aus .venv, eine Kernel-Auswahl ist nicht nötig. Das Terminal muss geöffnet bleiben, solange Sie arbeiten; beenden mit Strg+C (macOS: Ctrl+C). Öffnet sich kein Browserfenster, die im Terminal angezeigte Adresse (beginnt mit http://localhost:8888/, endet mit token=...) vollständig in den Browser kopieren.

Wöchentlicher Ablauf.

1. Vor jedem Lab neue Materialien holen: `git pull`, danach `uv sync` (installiert neu hinzugekommene Pakete).
2. Nicht im Starter-Notebook selbst arbeiten, sondern in einer Kopie, z. B. lab01_nachname.ipynb – sonst kollidieren Ihre Änderungen beim nächsten `git pull`.
3. Ergebnisse landen in results/. Ab dem Lab *CNNs & Computer Vision* baut jede Woche auf diesen Dateien auf (Trägermodell): nichts umbenennen oder löschen.

A.3 Schnelltest im Terminal

Vor setup_check.ipynb prüft eine Zeile im Ordner ake, ob die Umgebung grundsätzlich steht:

```
uv run python -c "import sys, torch; print(sys.version.split()[0], torch.__version__,
↳ torch.get_num_threads(), torch.cuda.is_available())"
```

Erwartung: Python ≥ 3.11 ; PyTorch-Version unter Windows/Linux mit Endung +cpu (macOS ohne Zusatz); Thread-Zahl ≥ 4 ; False für CUDA – das ist korrekt und kein Fehler.

A.4 Häufige Probleme

Symptom	Lösung
uv nach Installation nicht gefunden	Terminal schließen und neu öffnen; unter Windows ggf. ab- und wieder anmelden.
python öffnet unter Windows den Microsoft Store	Immer <code>uv run python ...</code> verwenden oder unter <i>Einstellungen</i> → <i>Apps</i> → <i>App-Ausführungsalias</i> die Python-Einträge deaktivieren.
uv sync bricht mit Netzwerk-/SSL-Fehler ab PyTorch-Download $\approx 2,5$ GB statt ≈ 200 MB	Proxy: ins eduroam bzw. Heimnetz wechseln oder Umgebungsvariable <code>HTTPS_PROXY=http://<proxy>:<port></code> setzen. PyTorch wurde per <code>pip install torch</code> nachinstalliert: <code>.venv</code> löschen, <code>uv sync</code> erneut ausführen.
Browser zeigt nach <code>jupyter notebook</code> nichts an	Adresse mit <code>token=...</code> aus dem Terminal vollständig kopieren; läuft schon eine Instanz, wählt Jupyter einen anderen Port (8889 usw.).
<code>FileNotFoundError</code> beim Laden der Daten	Notebook liegt außerhalb des geklonten Repositorys (z. B. in <i>Downloads</i>) oder <code>data/</code> wurde verändert: Notebook im Repository öffnen; veränderte Daten mit <code>git restore data/</code> zurücksetzen.
Kernel stürzt beim Training ab (<i>Kernel died</i>)	Arbeitsspeicher voll: Browser und andere Programme schließen, Kernel neu starten; für die Interpretation die Logs aus <code>results_referenz/</code> nutzen.
Datei gesperrt, Training extrem langsam	Repository liegt in OneDrive/iCloud: nach <code>C:\Users\<name>\ake</code> bzw. <code>~/ake</code> verschieben.
Konflikt bei <code>git pull</code>	Eigene Änderungen in eine Kopie übernehmen, dann <code>git restore <datei></code> und erneut <code>git pull</code> .

Notlösung Google Colab. Nur übergangsweise, falls die lokale Einrichtung trotz der Tabelle scheitert, bis das Problem in der Sprechstunde gelöst ist; freiwillig und mit eigenem Google-Konto. Den kompletten Ordner

ake (inklusive data/) in Google Drive hochladen, Notebook in Colab öffnen und *vor allen anderen Zellen* ausführen:

```
from google.colab import drive
drive.mount("/content/drive")
%cd /content/drive/MyDrive/ake/labs/woche01
```

Colab nutzt andere Paketversionen als die Lock-Datei (Ergebnisse können leicht abweichen), beendet die Laufzeit nach Inaktivität und unterstützt kein Docker.